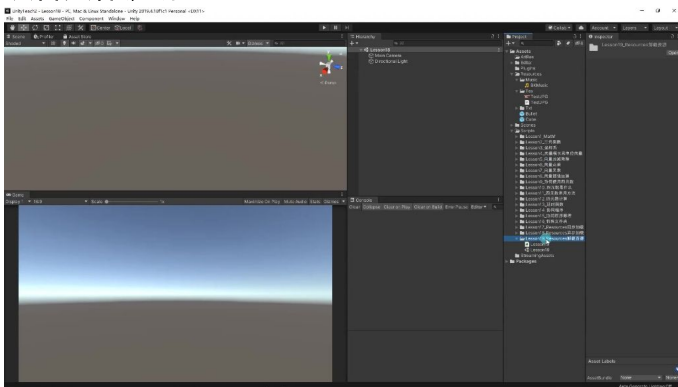


一、资源卸载 00:00

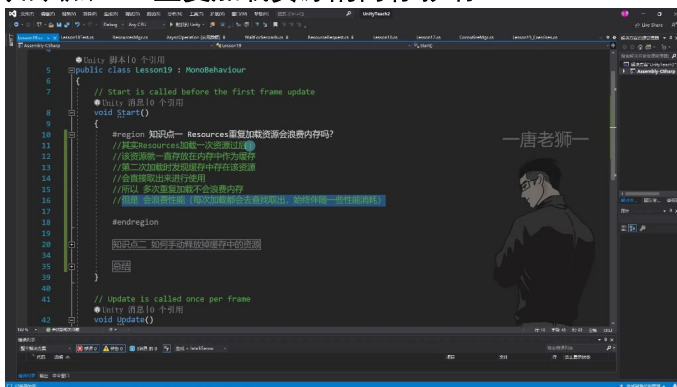


一、项目文件说明 00:12



- 新建文件夹：需要在Unity工程中新建Lesson19文件夹
- 脚本准备：新建脚本和场景用于演示资源卸载功能

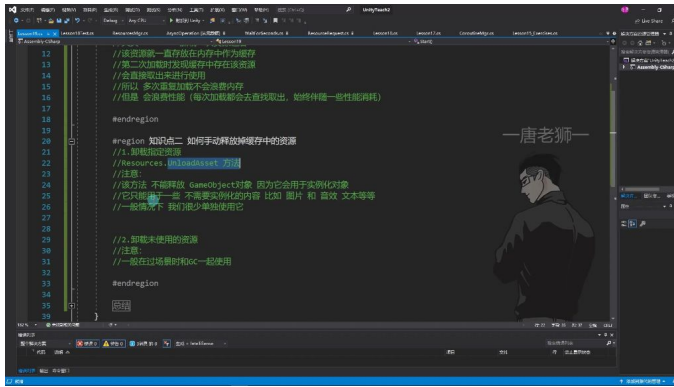
一、知识点一：重复加载资源的内存影响 00:24



- 缓存机制：Resources加载一次资源后，该资源会一直存放在内存中作为缓存
- 重复加载特性：
 - 第二次加载时发现缓存中存在该资源会直接取出使用
 - 多次重复加载不会浪费内存
- 性能消耗：每次加载都会去查找取出，始终伴随一些性能消耗

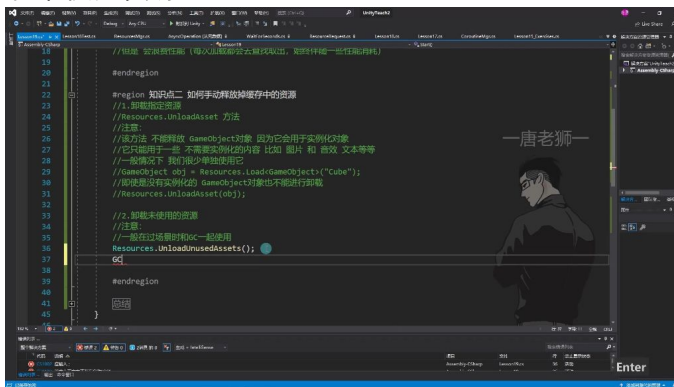
一、知识点二：手动释放缓存资源 01:14

3. 卸载指定资源



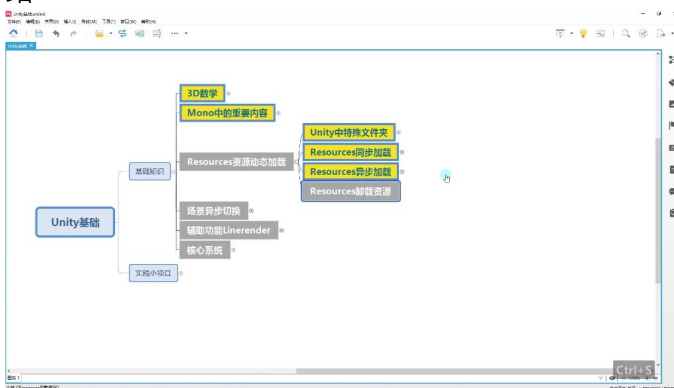
-
- 方法：使用Resources.UnloadAsset()
- 限制：
 - 不能释放GameObject对象(会用于实例化)
 - 只能用于不需要实例化的内容(图片、音效、文本等)
- 使用频率：一般情况下很少单独使用
- 示例验证：
 - 加载Cube预设体会报错，验证GameObject无法卸载
 - 图片资源可通过内存调试窗口观察卸载效果

3. 卸载未使用资源



-
- 方法：Resources.UnloadUnusedAssets()
- 使用时机：
 - 一般在过场景时使用
 - 需配合GC.Collect()一起使用
- 效果：执行后会把所有没用到的资源全部释放
- 内存验证：
 - 通过Unity Profiler窗口观察内存变化
 - 按1键加载图片，按2键卸载，内存有明显增减

4. 总结 09:28



- 指定资源卸载：
 - Resources.UnloadAsset卸载指定资源
 - 不能卸载GameObject对象
- 批量资源卸载：
 - Resources.UnloadUnusedAssets卸载所有未使用资源
 - 一般过场景时配合GC使用
- 重复加载验证：
 - 多次按1键加载同一资源不会增加内存占用
 - 按2键卸载后内存才会释放

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
Resources重复加载资源的内存影响	重复加载不会浪费内存（资源缓存），但会浪费性能（每次加载需查找）	缓存机制 vs 性能消耗	★★
手动释放缓存资源的方法	1. Resources.UnloadAsset(): 卸载指定资源（不支持GameObject, 仅限图片/音效/文本等） 2. Resources.UnloadUnusedAssets() + GC.Collect(): 卸载未使用资源（场景切换时常用）	GameObject释放限制 vs 非实例化资源释放	★★★
内存验证实验	通过Unity调试窗口（Ctrl+7）监测内存变化： - 加载图片后内存增加 - 卸载后内存恢复 - 重复加载不增加内存	内存监控工具使用	★★
资源卸载注意事项	- UnloadAsset不能用于脚本或GameObject - 卸载后需置空引用避免错误	API适用场景区分	★★★